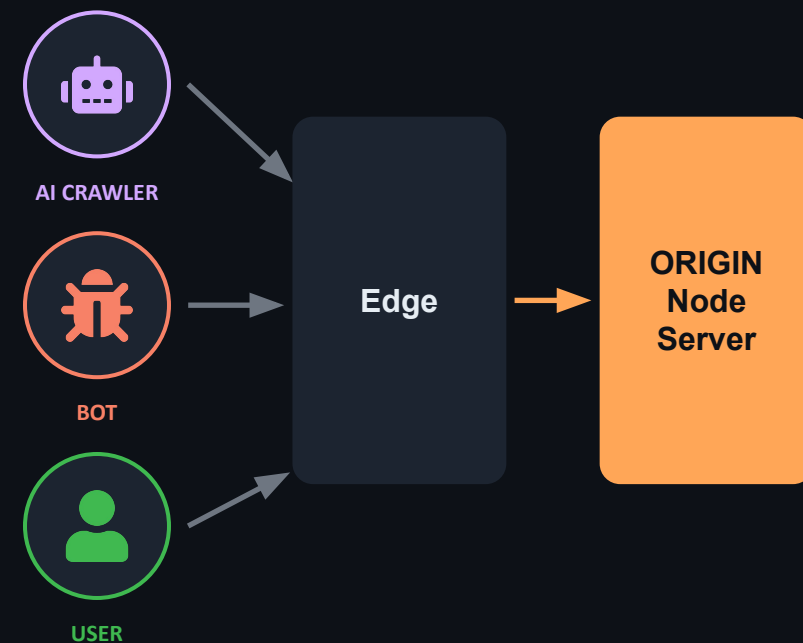


Your Server-Rendered App Doesn't Know Who It's Talking To

Traffic-aware SSR patterns and differential bot/human caching that cut a production Node fleet by ~80%.

Anshul Bansal

Principal Engineer, OLX India · 35M monthly users



untyped traffic → high CPU toll

THE PROBLEM

A backlog item we never prioritised... till now

Context

There was **no outage**. There was **no alert**.
Just a silent ticket that had sat in our backlog for a while...

The Symptoms

Divergent Metrics

Cost was climbing faster than users. Every capacity graph showed a heavier load, but none of them could explain why.

The Phantom Driver

Something invisible was steering the infrastructure and driving the server fleet upward, completely independent of human users.



Backlog Ticket

Description “Pod count looks high for the traffic we serve”

Pods running **»** **traffic served**

Cost **▲** · Users **→**

Who's actually asking?

Step 1 — what is Node doing? Two jobs on one single-threaded server:

80%

API proxy calls

pass-through, cheap per request — but they clog the event loop

20%

SSR page renders

CPU-bound, the expensive operation — the one we profiled next

Step 2 — inside that 20%, who is asking? SSR requests by user agent, New Relic:

75%

bots

search crawlers · AI crawlers · ad-platform scrapers

of the most expensive thing we do, three-quarters was for machines

25%

real users

**We weren't running a large fleet for our
users.**

**We were running it for SEO and AI
crawlers**

The autoscaling ratchet

HPA, simplified

```
minReplicas: X
maxReplicas: Y
target cpu: 60%
```

```
scaleUp:                                # aggressive
  stabilization: 60s
  +50% per 30s or +10 pods per 15s
  selectPolicy: Max                    # whichever is bigger
```

```
scaleDown:                              # conservative
  stabilization: 240s
  -30% per 5min or -10 pods per 60s
  selectPolicy: Min                    # whichever is smaller
```

Pod count over time (illustrative)



Up in seconds

A bot surge across thousands of URLs pushes CPU past 40%. Within a minute, +50% or +10 pods — whichever is more.

Down in minutes

Four minutes of calm before any scale-down, then -30% per 5 min or -10 pods per min — whichever is less.

The next surge lands before scale-down finishes. The floor only ever moves one way.

Three-phase edge load shaping

1



Offload proxy calls

Divert unnecessary API proxy traffic away from origin

2



Block bad traffic

Stop invalid and spam requests at the edge, before origin

3



Differential caching

Cache by traffic type — biggest impact of the three

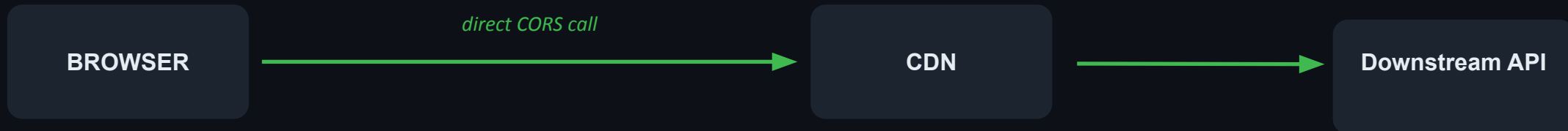
Our edge happens to be Akamai — every pattern here applies to any CDN.

Offloading API proxy calls

BEFORE



AFTER



High-traffic endpoints moved away from Node.js pass-through processing — the render tier stops paying for calls it never needed to see.

Three layers of edge defense



Rate-limit/Block by user-agent

Ad-platform scrapers added load and no value. Capped at the edge — origin never sees the burst.

`burst → capped`



Block spam by URL pattern

Campaigns injected junk into search paths. Structural patterns (not keyword lists) match them and serve a cached 410.

`/search/<junk> → 410`



Block by TLS fingerprint

Spoofed user-agents get past UA rules. Network/TLS fingerprints don't care what a client claims to be.

`one fingerprint = huge fake volume`

Together: millions of invalid requests a day stopped before origin — each one an edge rule instead of an origin failure.

Differential caching by traffic type

Humans and bots need different content freshness. So give each audience its own TTL.

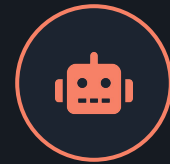


Humans

TTL:

minutes

Viewing live listings — need current prices, availability, state



Bots & crawlers

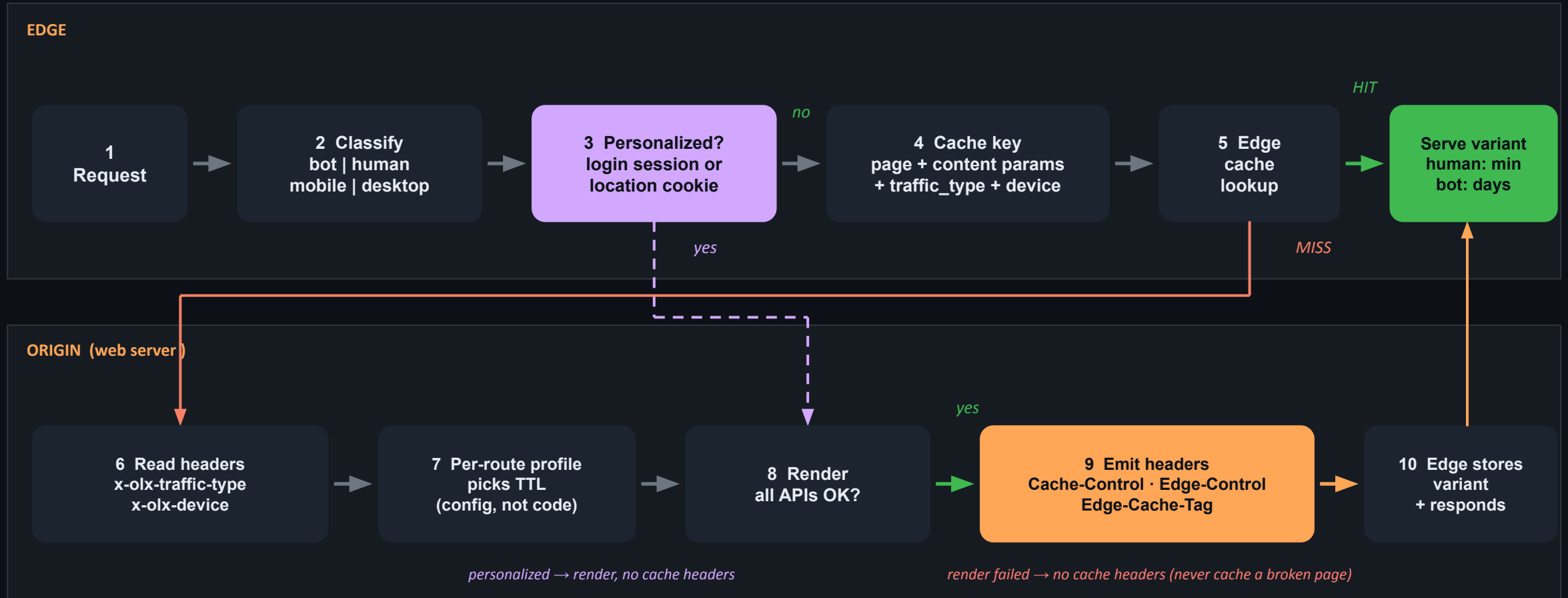
TTL:

days

Indexing content — a page cached for hours or days is perfectly fine

Device type is detected at the edge before the cache lookup — mobile and desktop are different builds.

Differential caching, end to end



Listing changes → purge by Edge-Cache-Tag, no waiting for TTL · Missing headers → treated as human (fail toward fresh)

How the classification reaches origin

```
const TRAFFIC_TYPE_HEADER = 'traffic-type';
const DEVICE_HEADER      = 'device';

const middleware = logger => (req, res, next) => {
  const trafficType = req.headers[TRAFFIC_TYPE_HEADER];
  const device = req.headers[DEVICE_HEADER]?.toLowerCase().trim();

  if (trafficType) req.isBot = trafficType.toLowerCase() === 'bot';
  if (CANONICAL_DEVICES.has(device)) req.device = device;

  nr.addAttribute('traffictype', trafficType || 'human', true);
  nr.addAttribute('devicetype', req.device || '', true);

  return next();
};
```



Two headers, set at the edge

x-olx-traffic-type and x-olx-device arrive before Node does any work



Absent header → human

The safe default fails toward freshness, never toward a stale page



Device fallback

No device header? User-agent detection still runs



Observability closes the loop

Traffic type becomes a New Relic attribute on every transaction

From traffic type to cache headers

per-route config (example: listing page)

```
human      max-age  minutes
bot        max-age  days
cacheTag    listing:<id>
```

```
// SSR handler — only after every upstream API succeeded
const profile = options[req.isBot ? 'bot' : 'human'];
res.setHeader('Cache-Control', `public, max-age=${maxAge}`);
res.setHeader('Edge-Control', `public, max-age=${maxAge}`);
if (cacheTag) res.setHeader('Edge-Cache-Tag', cacheTag);
```



Config, not code

Same route, two lifetimes, chosen by who's asking. A TTL change is a config change.



Cache only on success

Headers go out only if every API behind the render passed. A degraded page never gets cached for a day.

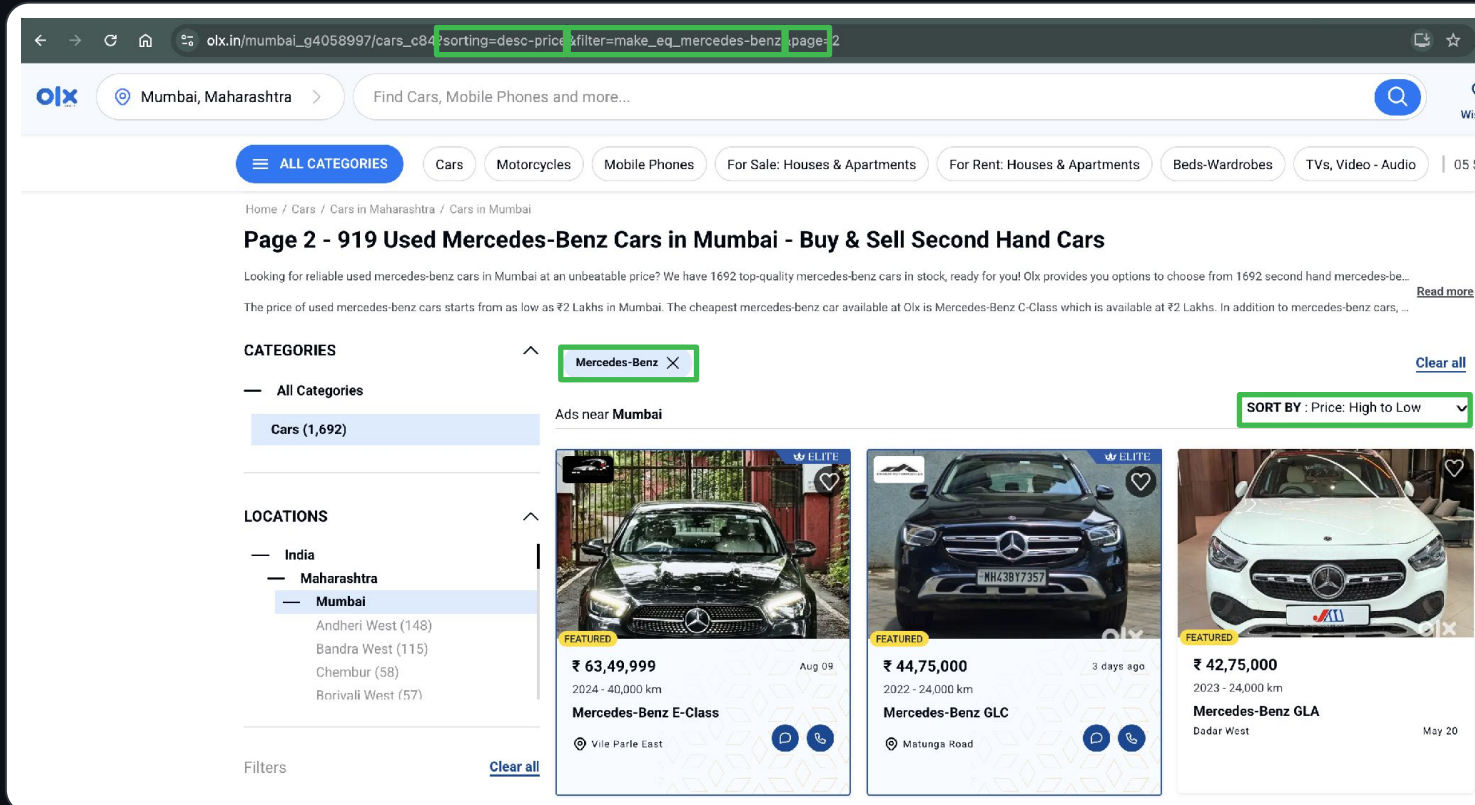


Tag every cached page

Edge-Cache-Tag → purge by tag the moment a listing changes. Invalidation doesn't wait for TTL.

Edge-Control / Edge-Cache-Tag are our edge's names — Surrogate-Control or CDN-Cache-Control elsewhere.

Edge Cache Key Creation



olx.in/mumbai_g4058997/cars_c84?sorting=desc-price&filter=make_eq_mercedes-benz&page=2

cache_key = page + content_params + traffic_type + device_type

content params — in the key
filters · pagination · sort · language

traffic_type

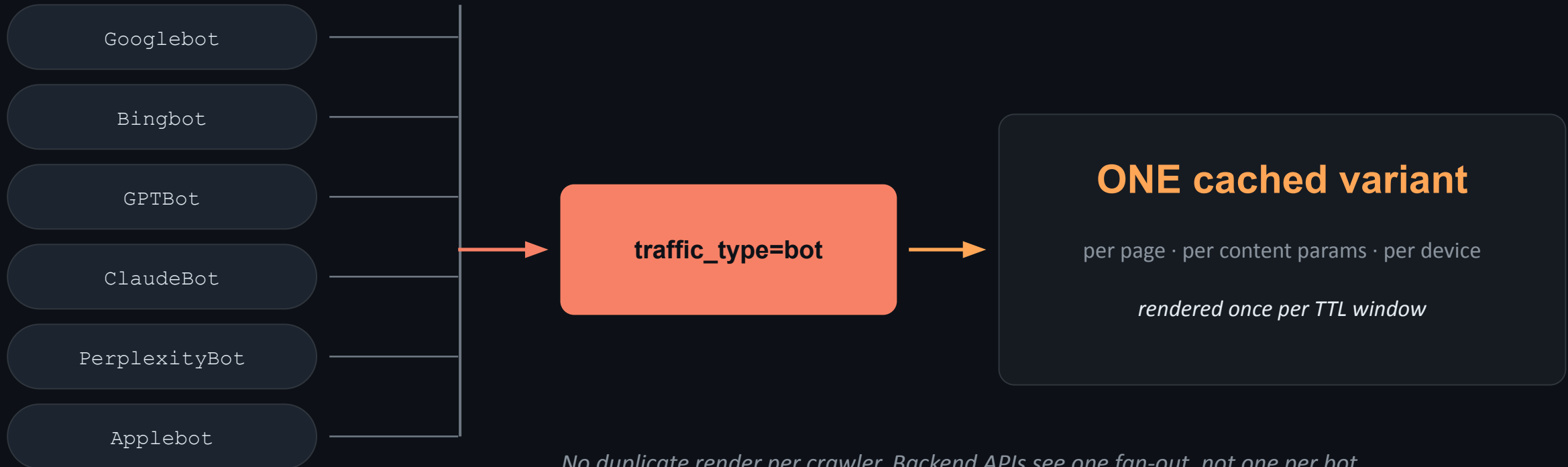
human | bot — every crawler
variant collapses into one value,
sharing one cached copy

device_type

mobile | desktop — separate builds,
detected before lookup

Bot cache consolidation

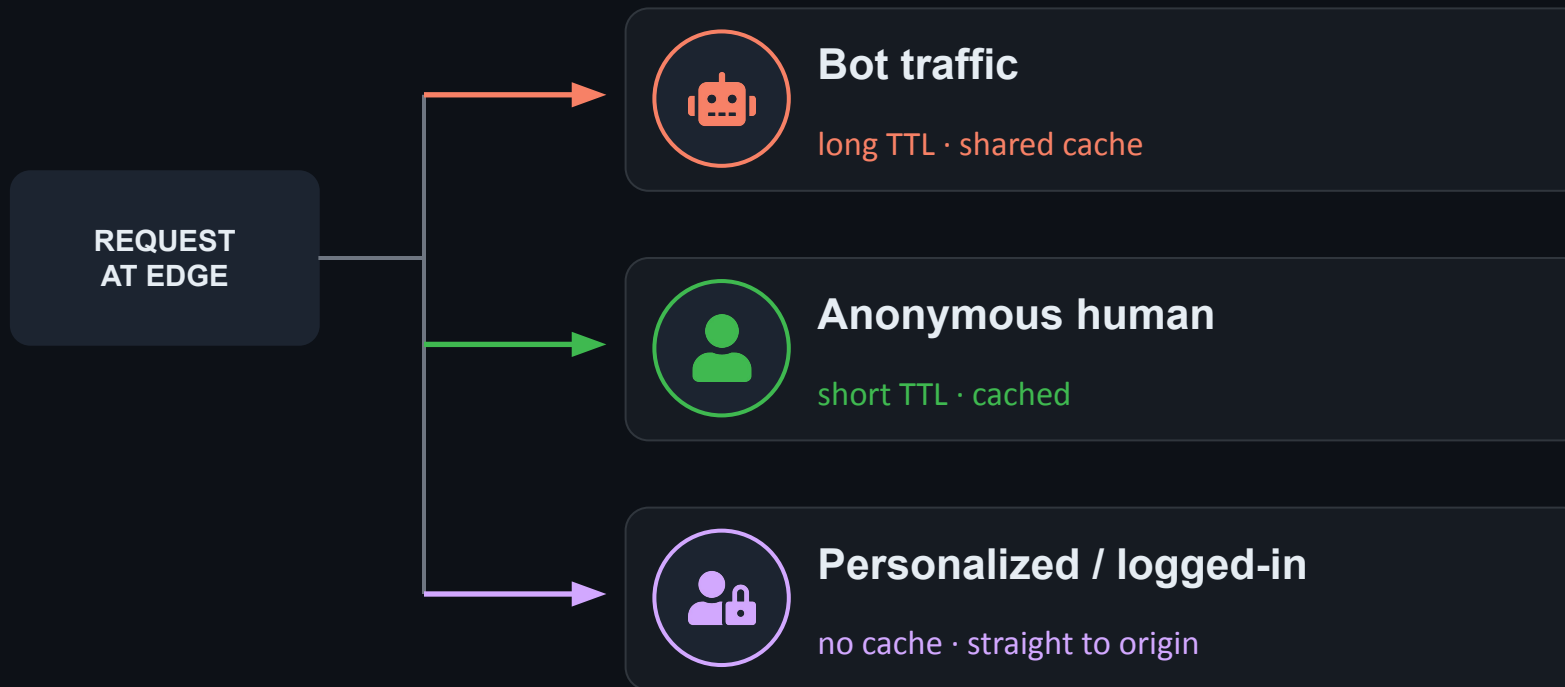
Every valid search-engine crawler and AI scraper maps to one value. They all share one cached page variant.



No duplicate render per crawler. Backend APIs see one fan-out, not one per bot.

Separating personalized & logged-in traffic

Two signals mark a request as personalized: an active login session, or a location cookie. That traffic bypasses the cache entirely.



Why not key on user ID?

- One cache entry per user — caching becomes nearly pointless
- Any key bug could serve user A's page to user B — a correctness bug that's also a privacy bug
- Bypassing removes the whole failure class

Good bots vs. bad bots



Before

- Blocked some AI crawlers outright — infrastructure couldn't absorb the load
- As AI search grew, that became a visibility risk: invisible to AI-powered experiences

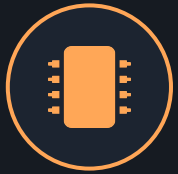


After

- Caching made the traffic affordable
- Good bots can crawl as much as they like — but only hit origin once per cache window
- Bad bots are still blocked at the edge

Fewer, fatter pods

Edge changes cut what reached origin. Origin changes made each remaining pod worth more. The two compounded.



2× CPU per pod

SSR is CPU-bound. Doubling CPU per pod let each one carry far more rendering work than two smaller pods — fewer cold starts, less scheduler churn.



Off the slow node generations

Moved workloads away from node types and generations that showed higher latency.



Rebalanced across zones

Refined how workloads were distributed across availability zones for stability under surge.

Less traffic reached origin × each pod handled more → the 80% is the product, not the sum.

Cache hit ratio: average 30% → 55–60%



Edge cache hit ratio, www.olx.in, 30 days around the rollout

Before

20–40%, swinging daily — every crawler variant and tracking param was its own cache miss

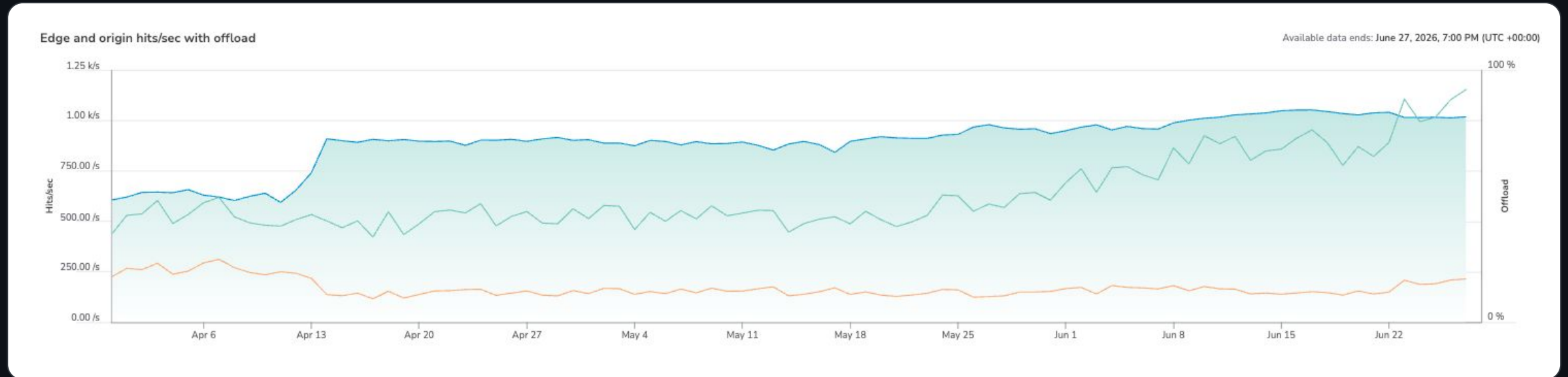
The step

One day. That's the differential-caching + cache-key rollout going live

After

55–60%. The shape matters as much as the number — predictable origin load

When the edge surges, origin doesn't



Edge vs origin hits/sec with offload, Apr → Jun 2026



Edge traffic keeps climbing

~450/s → ~1,100/s over the quarter, including surges



Origin stays flat

~150/s the whole way — the surges never reach it



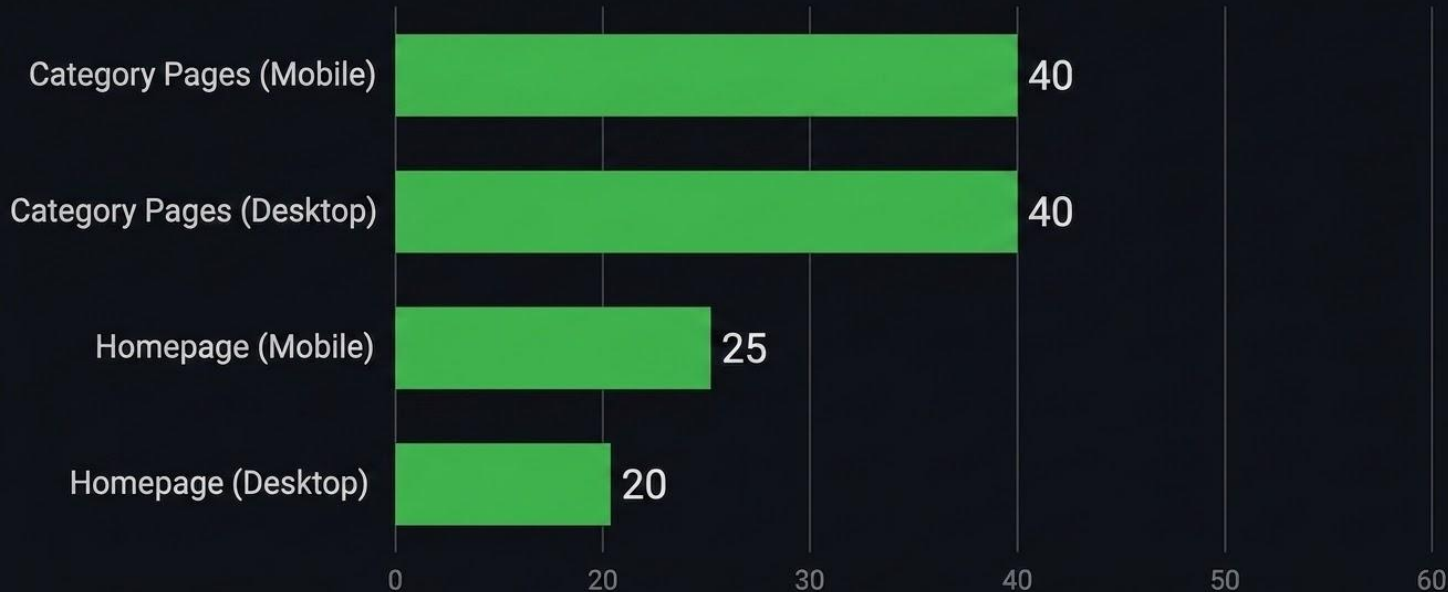
Offload holds ~85–90%

Step up at rollout, then it just stays there

Performance: faster pages for everyone

WEB PERFORMANCE IMPROVEMENTS (approx. %)

REFLECTS TTFB DOWN ~40–48% with more requests served at the edge
Largest Contentful Paint followed the same trend



- TTFB down ~40–48% with more requests served at the edge
- Largest Contentful Paint followed the same trend
- Category pages ~40% faster on mobile and desktop
- Homepage ~20–25% faster

A growth story, not just a cost story

10–12%

daily active user growth contribution

from an infrastructure project that never touched the product



SEO & crawl budget

- Junk URLs blocked at the edge → crawl budget went to real, indexable pages instead of spam
- Faster pages improved the experience for users and search alike
- Good AI crawlers welcomed back — visibility in AI-powered search at near-zero origin cost

Building for AI traffic

The same traffic strategy becomes the foundation for welcoming AI crawlers without paying for them twice.



lms.txt & robots.txt

Declare what AI crawlers can access and where the useful content lives



Markdown content negotiation

Serve a lean, structured representation to agents instead of a full HTML render



Warm-cache serving

AI crawlers hit the shared bot cache — visibility in AI-powered experiences at near-zero origin cost

Four architectural rules for edge SSR platforms

1

Profile the client, not just the path

Know who is asking before deciding what a request costs you

2

Implement differential caching

Different audiences need different freshness — give them different TTLs

3

Protect the autoscaling floor

Never let bot surges set your baseline capacity

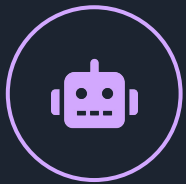
4

Consolidate cache keys

Whitelist what changes the page; collapse crawlers into one bucket

*We weren't running a large fleet because we had too much traffic.
We just didn't know who our traffic was.*

Once we did, the caching, the capacity, the cost — all of it followed on its own.



This problem isn't going away — it's getting more interesting.

"Who is this request from" used to mean search crawlers and scrapers. Increasingly it means autonomous agents acting on someone's behalf. If your app assumes every request is a human at a browser, you'll hit the same wall — just faster.

Do you actually know who's asking?



Do you know how your app behaves during off-peak automated crawls?



Are you rendering CPU-intensive SSR payloads to crawlers that could be served from the edge?

Questions?

Scan the QR code to connect, or let's discuss:

DISCUSSION TOPICS

- **Edge SSR Architecture** & cache optimization
- **AI Bot Mitigation** & traffic strategies
- **Scaling Web Platforms** to millions of users



Anshul Bansal

Building the web platform for OLX India's 35M users ·
Principal Engineer, Frontend Infrastructure | PWA ·...

